One Does Not Simply **Query** a Stream

A landscape guide to querying your Kafka data

Viktor Gamov

Who's Here Tonight?

- How many of you are **using Kafka** in any capacity? (raise your hand)
- Keep your hands up if you're doing **Kafka Streams** or **Flink**
- How many of you have tried to **query** data from Kafka? (like, SQL-style)
- How many gave up and just dumped it into Postgres? (no judgment... okay, a little judgment)

How This Talk Was Born

A couple of years ago, someone asked on Twitter...

> "When should I use Kafka Streams vs a real-time analytical database like Pinot?"

And my boss said...

"That might be a very good idea for a talk."

It turns out to be a **very deep** rabbit hole.

Today you get the **compacted version**. Kind of like a distilled version.

Before We Start

- Slides and recording at **speaking.gamov.io**
- I'm Viktor Gamov
- Developer Advocate at **Confluent**
- Co-author of Kafka in Action (O'Reilly)
- Java Champion

Find me:

- X/Bluesky: **@gamussa**
- GitHub: **gAmUssA**
- gamov.dev/rel

The Problem

Simpler Times

Once upon a time, you had a **monolith**.

One application. One database. One SQL query.

```
SELECT * FROM orders WHERE status = 'pending';
```

Life was good. You could query anything.

Then someone said "microservices" and it all went sideways.

All Roads Lead to Kafka

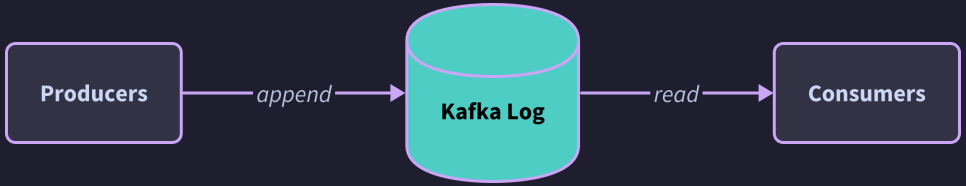
Your data is already in Kafka topics.

- Events flow through topics in real time
- Data is **immutable** — once it's written, it's written
- Kafka is an append-only log, not a database

So how do you **query** it?

You can't just `SELECT * FROM kafka_topic`. (Well... actually... we'll get to that.)

The Log



Think of it like a **ledger**. You can add entries. You can read entries.

You cannot update entries. Kind of like a conversation with your significant other.

OLTP vs OLAP

This is the most important slide in this talk.

OLTP

Transactional queries

- Get me order #12345
- What's the current balance?
- Point lookups by key
- **Low latency, single record**

OLAP

Analytical queries

- How many orders this hour?
- What's the average basket size?
- Aggregations across millions
- **Higher latency, many records**

Every solution we'll see today optimizes for one of these. Keep this in your head.

The Usual Suspects

Our Options Tonight

---

1. **Kafka Connect** + Relational Database
2. **Kafka Streams** (embedded querying)
3. **Streaming SQL** databases
4. **Real-Time OLAP** databases
5. **Elasticsearch**
6. **Cloud Data Warehouses**
7. **Data Lakes** + Table Formats
8. **Tableflow** (the new kid)

I will **not** give you a definitive answer. There are no right solutions. Only trade-offs.

Option 1: Kafka Connect + RDBMS

Connect + Database

The "just dump it into Postgres" approach.

When Connect + RDBMS Works

- + You already know SQL
- + Familiar tooling (pgAdmin, DBeaver, all this type of jazz)
- + Great for **smaller datasets**
- + OLTP-friendly — point lookups by key
- - Not real-time — there's a **lag** between Kafka and DB
- - Doesn't scale to millions of events/sec
- - You're maintaining another database

If this works for you? **It's fine.** It's totally fine. This is not a bad thing.

Option 2: Kafka Streams

Kafka Streams

What if you could query the stream **from inside your application**?

```
StreamsBuilder builder = new StreamsBuilder();
KTable<String, Long> counts = builder
    .stream("page-views")
    .groupByKey()
    .count();
// Interactive Queries - the OLTP trick
ReadOnlyKeyValueStore<String, Long> store =
    streams.store(StoreQueryParameters.fromNameAndType(
        "counts-store", QueryableStoreTypes.keyValueStore()));
Long count = store.get("home-page");
```

Kafka Streams: The Baby Bird Analogy

How does a **consumer group** work?

Imagine you have a nest with baby birds.

Mommy bird brings **worms** (events). Each baby bird (consumer) gets a worm.

What happens if another baby bird **hatches** and there are 4 worms but 5 baby birds?

> "It dies."

Not entirely. It will be **hungry**. It will starve. But rebalancing will happen.

When Kafka Streams Works

- + No external database needed
- + Embedded in your Java/Kotlin app
- + Interactive Queries for **OLTP** lookups
- + Exactly-once processing guarantees
- - It's a **library**, not a service — you manage the deployment
- - Analytical queries (OLAP) are limited
- - Scaling means scaling your app instances

Congratulations, you built your own database.

Option 3: Streaming SQL

What's a Streaming Database?

- Takes a **streaming source** (Kafka) as first-class input
- Exposes a **SQL interface** for queries
- Maintains **persistent state** — materialized views updated continuously

The dream: write SQL, get real-time results.

The Candidates

- ~~ksqlDB~~ (we're not talking about this guy anymore)
- **RisingWave** — "Flink in Rust" (sue me)
- **Timeplus** — streaming analytics focus

R.I.P.

ksqlDB
2018-
2024

"But Viktor, Flink Has SQL!"

Yes. Yes it does.

And Flink SQL is **excellent** for building streaming pipelines.

But Flink is a **processing framework**, not a database.

It doesn't serve queries. It processes them and writes results somewhere else.

Different tool, different job.

RisingWave

- Distributed SQL streaming database
- Built in **Rust** (the cool kids rejoice)
- PostgreSQL-compatible interface
- Kafka as first-class streaming source
- Connectors for Iceberg, S3, DynamoDB

Think of it as... kind of like Flink reimagined as a database.

When Streaming SQL Works

- + SQL interface — familiar for most developers
- + Continuous materialized views — OLAP on streams
- + No custom code needed
- - Another service to deploy and manage
- - Scaling characteristics vary wildly between products
- - Community support is... developing

Option 4: Real-Time OLAP

Real-Time Analytical Databases

What if you need **sub-second** queries over **billions** of events?

The Players

- **Apache Pinot** (I used to build this)
- **Apache Druid**
- **StarRocks**

Characteristics

- High concurrency queries
- Ultra-low latency
- **Native Kafka ingestion**
- Purpose-built for OLAP

Real-Time OLAP: When It Shines

Think **LinkedIn's "Who Viewed Your Profile"** — millions of users querying real-time analytics simultaneously.

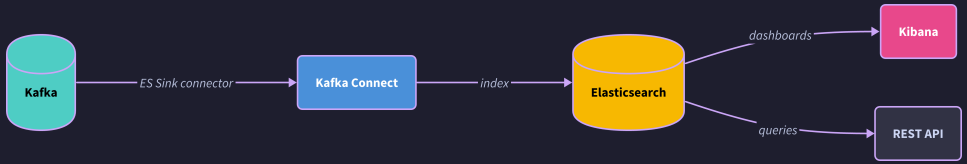
- + **Millisecond** query latency at massive scale
- + Built for concurrent analytical queries
- + Kafka is a first-class data source
- - Specialized — you're adding a dedicated OLAP cluster
- - Complex operational overhead
- - Schema management can be... interesting

Anyone using Apache Druid? StarRocks?

Option 5: Elasticsearch

Elasticsearch + Kafka

Kafka → Elasticsearch — the search and analytics pipeline.



Pipeline: Kafka Connect Elasticsearch Sink connector — battle-tested ingestion from streams.

Analytics: Elasticsearch as a hybrid search + analytics engine — full-text search, aggregations, and Kibana dashboards out of the box.

When Elasticsearch Works

- + **Full-text search** — something none of the other options do well
- + Kibana for dashboards and exploration out of the box
- + Kafka Connect sink connector is battle-tested
- + Hybrid: both point lookups (OLTP-ish) and aggregations (OLAP-ish)
- - Not a streaming engine — it's a destination, not a processor
- - Schema mapping can get tricky with nested Avro/JSON
- - Cluster sizing and shard management at scale

But you all know this already. That's why you're here tonight.

Option 6: Cloud Data Warehouses

Cloud DWH + Kafka

- + Massive scale, managed service
- + SQL interface everyone knows
- + Kafka connectors available
- - **Batch-oriented** — even "streaming" modes have latency
- - Expensive at high volume
- - Structured data bias — semi-structured gets messy

Good for analytics. Not great for real-time.

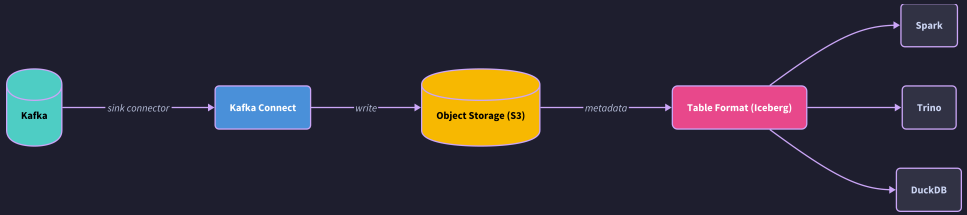
Option 7: Data Lakes + Table Formats

The Data Lake Story

"Just dump everything into S3. We'll figure it out later."

Later never came.

Then table formats appeared and saved us from ourselves:



Data Lake Formats

- **Apache Iceberg** — the one winning the adoption race
- **Apache Hudi** — Netflix's baby
- **Delta Lake** — Databricks' entry

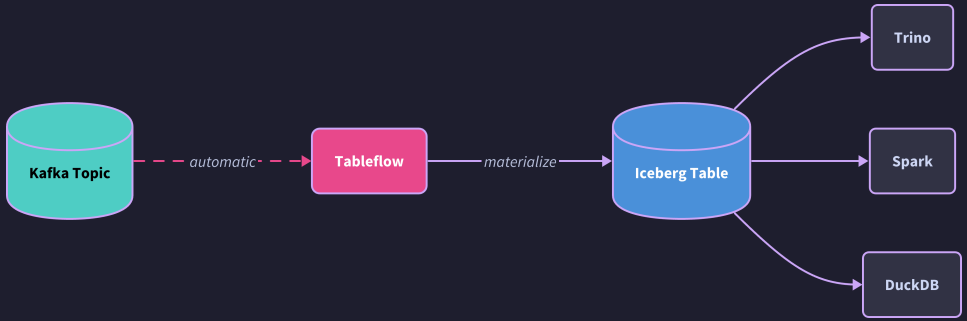
Data Lakes: The Good and the Ugly

- + Decoupled storage and compute
- + Open table formats (Iceberg!) — no vendor lock-in
- + Query with anything — Spark, Trino, DuckDB, Flink
- - **Not real-time** — compaction cycles mean minutes to hours of delay
- - Complex infrastructure to maintain
- - Getting streaming data IN is the hard part

Option 8: Tableflow

Tableflow: Kafka Topics as Iceberg Tables

What if your Kafka topic **was already** an Iceberg table?



No connectors. No ETL. The topic is the table.

When Tableflow Works

- + Zero infrastructure to manage
- + Open format (Iceberg) — query with any engine
- + Automatic schema evolution from Schema Registry
- + Data stays in your cloud storage
- - **Not real-time** — compaction has latency (minutes)
- - Confluent Cloud only (for now)
- - Read-only — you query the table, not the stream

It's a bridge between streaming and lakehouse. Pretty cool, actually.

So... Which One?

The Decision Framework

No ideal solutions. Only trade-offs.

```
OLTP (point lookups)  →  Kafka Streams
                          Connect + RDBMS

OLAP (analytics)      →  Real-Time OLAP
                          Streaming SQL

Search + hybrid       →  Elasticsearch

Batch analytics       →  Data Lake
                          Cloud DWH
                          Tableflow
```

Three Things to Consider

1. **Familiarity** — sometimes you go with what you know. And that's not a bad thing.
2. **Performance** — if consumer lag keeps you up at night, look at Pinot or StarRocks
3. **Community** — when you're choosing a solution, think about where you can go ask questions

The Community Thing

When I was learning to program, I picked **Pascal**.

Not because it was the best language. Because there was a guy in my neighborhood who knew Pascal.

When I got stuck, I could walk over and ask him.

That's why Kafka won. Not because it's perfect.

Because **people at meetups like this one** could eat pizza and help each other figure it out.

What to Try This Week

1. **Already using Kafka?** Try Interactive Queries with Kafka Streams
2. **Want SQL on streams?** Spin up RisingWave or Materialize locally
3. **Need analytics at scale?** Look at Pinot + Kafka connector
4. **On Confluent Cloud?** Enable Tableflow on a topic and query with DuckDB
5. **Come talk to me** — I'll be around after. Pizza first.

Resources

- **Slides**: speaking.gamov.io
- **Streaming Frontiers**: Live stream series (every 4 weeks)
- **Confluent dev resources**: developer.confluent.io
- **Book**: Kafka in Action (Manning)

As always, have a nice day.

Viktor Gamov — X/Bluesky: @gamussa

Questions?

I'll be in the hallway track. Find me near the pizza.

X/Bluesky: @gamussa